

Week 8 - Multivariate Analysis

Please run the cells of the notebook as you get to them while reading

```
In [1]: import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
from sklearn import decomposition
from sklearn import linear_model
from sklearn import metrics
```

1. Lesson: Multivariate graphs

In this lesson, we'll consider some multivariate plots: heatmaps and bubble plots. We'll also consider principal component analysis and multiple regression.

Heatmaps

In this plot, we use a heatmap to count the total traffic tickets given to citizens. A heatmap requires the x and y axes to be categorical, while the color involves a third, numerical variable. In this example, each row of the DataFrame represents a traffic ticket given to one person in a particular city and year. The data are given as three columns (city, year, and ticket), so we must use a pivot_table to transform it into a two-dimensional grid. The cmap is set to "Blues" in the first example, meaning that the heatmap uses only blue as its color. However, we could also use a multicolor map like "viridis" in the second example. I personally think "Blues" is easier to read - what do you think? Here is a reference guide to the color maps:

https://matplotlib.org/stable/gallery/color/colormap_reference.html

```
In [2]: np.random.seed(0)
num_samples = 100
city_series = np.random.choice(["New York", "Boston", "Atlanta", "Washington, D. C.",
year_series = np.random.choice([2018, 2019, 2020], size = num_samples)
ticket_series = np.random.randint(50, 150, size = num_samples) # Traffic tickets ca
df = pd.DataFrame({"city": city_series, "year": year_series, "ticket": ticket_serie
pivot = df.pivot_table(index = ["city"], columns = ["year"], values = "ticket", agg
```

```
In [3]: df.iloc[0:5] # first five rows of a Long DataFrame
```

```
Out[3]:
```

	city	year	ticket
0	New York	2019	79
1	Washington, D. C.	2020	71
2	Boston	2018	75
3	New York	2018	130
4	Washington, D. C.	2020	110

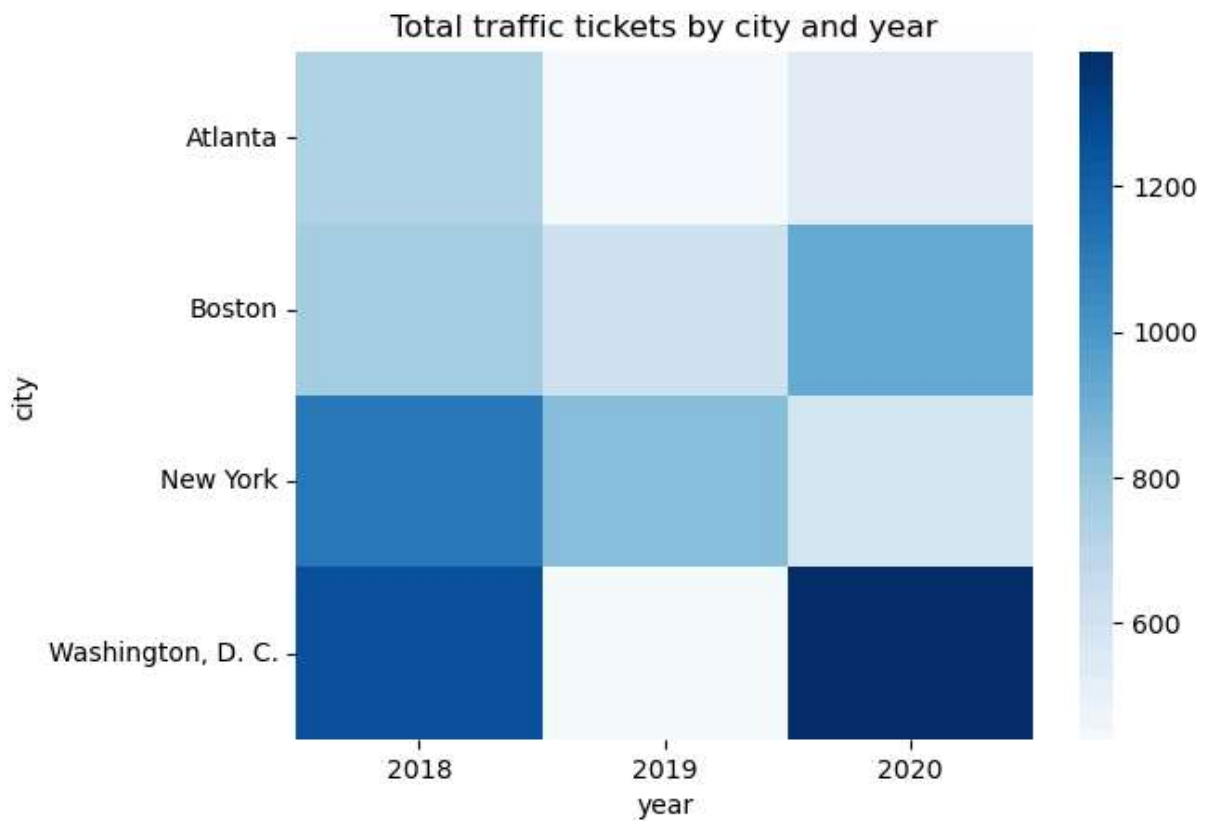
```
In [4]: pivot # 2 by 2 grid showing total ticket amounts
```

```
Out[4]:
```

	year	2018	2019	2020
	city			
	Atlanta	742	440	538
	Boston	775	628	922
	New York	1122	841	596
	Washington, D. C.	1258	451	1385

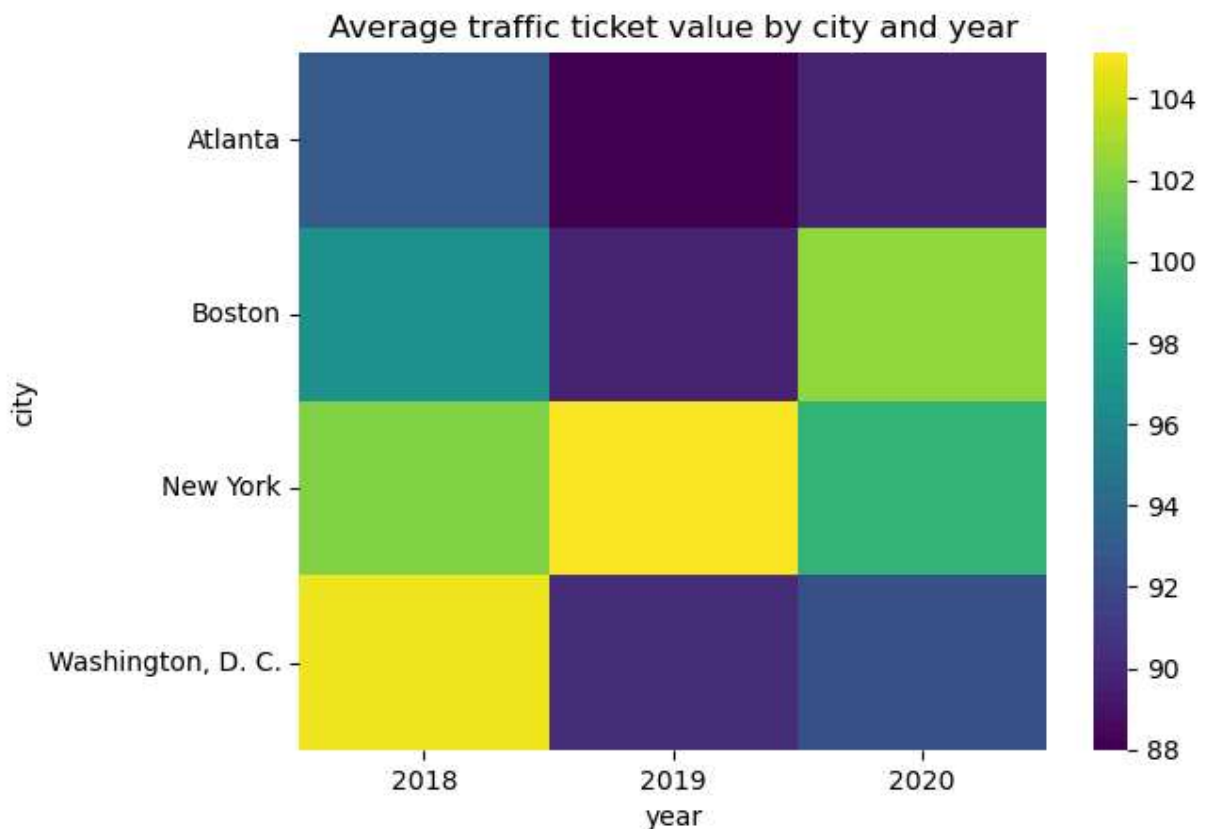
```
In [5]: ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total traffic tickets by city and year")
```

```
Out[5]: Text(0.5, 1.0, 'Total traffic tickets by city and year')
```



```
In [6]: pivot2 = df.pivot_table(index = ["city"], columns = ["year"], values = "ticket", ag
ax2 = sns.heatmap(pivot2, cmap = "viridis") # do you think this is a better or wors
ax2.set_title("Average traffic ticket value by city and year")
```

```
Out[6]: Text(0.5, 1.0, 'Average traffic ticket value by city and year')
```

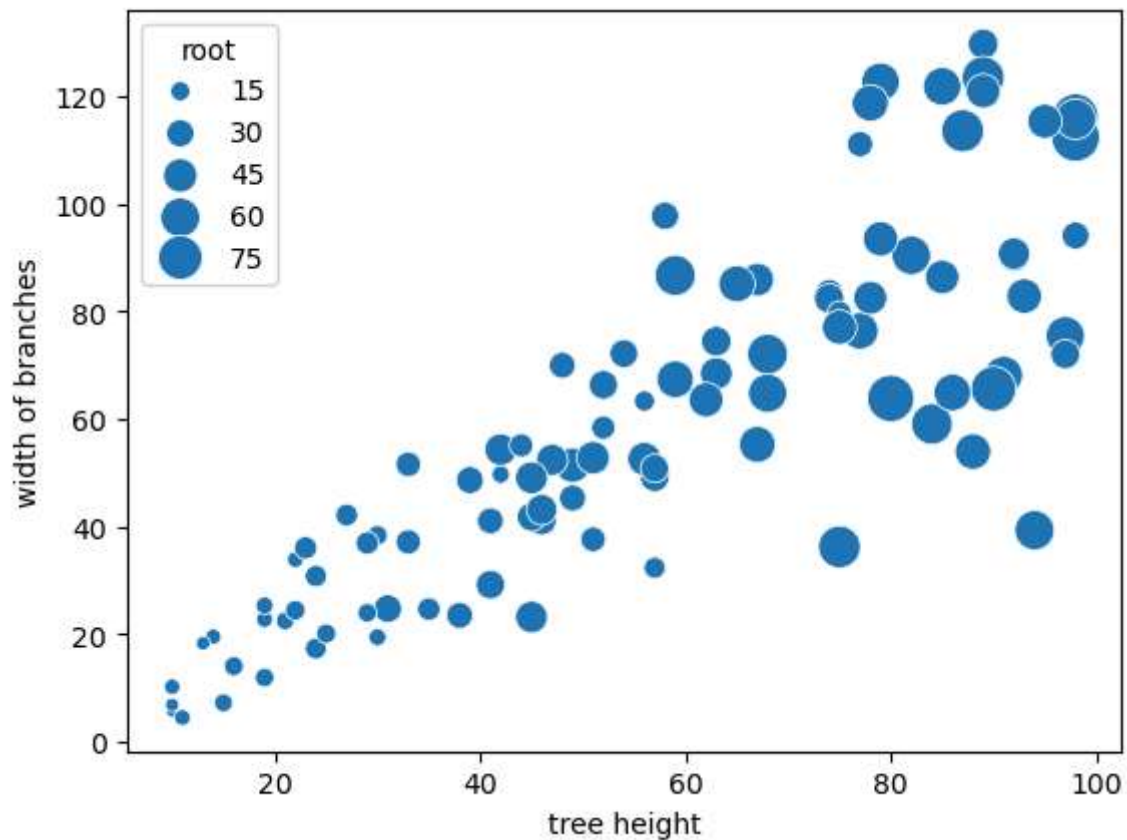


Bubble plots

A bubble plot is just a scatter plot, but with the dot size representing a third variable. Here, each data point shows a tree (I mean the kind of tree that is a plant and grows in the ground, not a decision tree), with its height, width of branches, and root depth. The bubble size in the bubble plot represents the root depth, while the other variables are shown on the x and y axes. From this plot, we can see that the root depth is strongly correlated with both the width of branches and the tree height.

```
In [7]: np.random.seed(0)
num_samples = 100
height_series = np.random.randint(10, 100, size = num_samples) # tree height
width_series = height_series * np.maximum(np.random.normal(scale = 0.3, loc = 1, si
root_series = height_series * np.maximum(np.random.normal(scale = 0.2, loc = 0.7, s
df2 = pd.DataFrame({'height': height_series, 'width': width_series, 'root': root_se
minroot = root_series.min()
maxroot = root_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "root", sizes =
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

```
Out[7]: Text(0, 0.5, 'width of branches')
```



Principal Component Analysis

Principal Component Analysis can be thought of as a way to use one (or a few) linear combinations of the coordinates in order to explain all of the coordinates. It is most useful when there are strong correlations between some or all of the coordinates. Principal Component Analysis can show us which linear combinations are most (and least) useful for explaining the data.

With the trees data, the three values (height, width, and root) are highly correlated. For this reason, Principal Component Analysis shows that most of the variance can be explained by a single value which is a linear combination of height, width, and root. What this means is that if the first components_ vector in the code below (the "first principal component") is $[a, b, c]$, for some specific numbers a , b , and c , then the value $v = a * (\text{height} - \mu_h) + b * (\text{width} - \mu_w) + c * (\text{root} - \mu_r)$, where the μ 's are the means, is highly informative about the values of height, width, and root. In this case, because height, width, and root are so highly correlated, we can almost assume that all of the data is located along an axis defined by the vector $[a, b, c]$. The point $v * [a, b, c] + (\text{mean vector}) = (a * (\text{height} - \mu_h) + b * (\text{width} - \mu_w) + c * (\text{root} - \mu_r)) * [a, b, c] + (\text{mean vector})$ is therefore approximately equal to the data point (height, width, root) in question. The mean vector is $[\mu_h, \mu_w, \mu_r]$. Importantly, there is still variance unexplained by this one value - just knowing $v * [a, b, c]$ does not tell you the height, width, and root with 100% accuracy. If the second principal

component is $[d, e, f]$, then we could approximate the tree vector even more accurately as:
 $(\text{height}, \text{width}, \text{root}) = (a * (\text{height} - \mu_h) + b * (\text{width} - \mu_w) + c * (\text{root} - \mu_r)) * [a, b, c]$
 $+ (d * (\text{height} - \mu_h) + e * (\text{width} - \mu_w) + f * (\text{root} - \mu_r)) * [d, e, f] + (\text{mean vector}).$

The printing of "components_", below, shows the first principal component $[a, b, c]$. If you draw a vector in the direction shown, you will be heading in the direction where the values height, width, and root change most rapidly. A later computation shows all three principal component vectors.

Optional exercise for the reader: show that if all three principal components are included, then equality is exact. This is going to be true simply because the principal components are orthonormal: the projections onto a complete set of orthonormal vectors always add up to the original vector.

$(\text{height}, \text{width}, \text{root}) = (a * (\text{height} - \mu_h) + b * (\text{width} - \mu_w) + c * (\text{root} - \mu_r)) * [a, b, c]$
 $+ (d * (\text{height} - \mu_h) + e * (\text{width} - \mu_w) + f * (\text{root} - \mu_r)) * [d, e, f] + (g * (\text{height} - \mu_h) + h * (\text{width} - \mu_w) + i * (\text{root} - \mu_r)) * [g, h, i] + [\mu_h, \mu_w, \mu_r].$

For this proof, we must use the fact that the principal component vectors are orthonormal. That is, any pair of them are orthogonal, even with a transpose ($ad + be + cf = 0$, $ag + bh + ci = 0$, $dg + eh + fi = 0$, $ab + de + gh = 0$, $ac + df + gi = 0$, $bc + ef + hi = 0$) and any one of them is a unit vector, even with a transpose ($a^2 + b^2 + c^2 = 1$, $d^2 + e^2 + f^2 = 1$, $g^2 + h^2 + i^2 = 1$, $a^2 + d^2 + g^2 = 1$, $b^2 + e^2 + h^2 = 1$, $c^2 + f^2 + i^2 = 1$). If you write out the equation above as a single vector, then apply the orthonormality rules, you should be able to prove equality.

```
In [8]: pca = decomposition.PCA(n_components = 1)
pca.fit(df2)
print(f"The first principal component explains {pca.explained_variance_ratio_[0]} of the variance")
print(f"The components_ vector is [a, b, c] = {pca.components_[0]}")
mu_h, mu_w, mu_r = df2.mean().values
print(f"(height, width, root) is approximately equal to ({pca.components_[0][0]:.2}, {pca.components_[0][1]:.2}, {pca.components_[0][2]:.2})")
print("Examples: (estimating the the vector equals its projection onto the first principal component)")
df_example = df2.copy()
df_example["v"] = pca.components_[0][0] * (df2["height"] - mu_h) + pca.components_[0][1] * (df2["width"] - mu_w) + pca.components_[0][2] * (df2["root"] - mu_r)
df_example["height_est"] = df_example["v"] * pca.components_[0][0] + mu_h
df_example["width_est"] = df_example["v"] * pca.components_[0][1] + mu_w
df_example["root_est"] = df_example["v"] * pca.components_[0][2] + mu_r
print(df_example.iloc[0:5])
print("Means:")
print(f"mu_h = {mu_h:.3}, mu_w = {mu_w:.3}, mu_r = {mu_r:.3}")
```

The first principal component explains 0.8686108480944676 of the variance
The components_ vector is [a, b, c] = [[0.59031629 0.71908728 0.36666083]]
(height, width, root) is approximately equal to $(0.59 * (\text{height} - 5.6e+01) + 0.72 * (\text{width} - 5.9e+01) + 0.37 * (\text{root} - 3.6e+01)) * (0.59, 0.72, 0.37) + (5.6e+01, 5.9e+01, 3.6e+01)$

Examples: (estimating the the vector equals its projection onto the first principal component)

	height	width	root	v	height_est	width_est	root_est
0	54	72.139334	31.517932	6.287751	60.001762	63.563941	38.673095
1	57	49.169527	33.768693	-7.633330	51.783921	53.553468	33.568780
2	74	83.443733	28.770850	25.215680	71.175227	77.174773	45.613225
3	77	76.353866	53.061241	30.794730	74.468631	81.186597	47.658844
4	77	111.095365	26.006307	45.856916	83.360084	92.017624	53.181557

Means:

mu_h = 56.3 mu_w = 59.0 mu_r = 36.4

If we were to choose a second or third component to preserve, that values below show that each would explain a small, additional amount of the variance. This means that with this particular dataset, we should likely do PCA using only one component and not two. If the components' explained variances were closer in magnitude to each other, we might have to include two or even all three values to explain the data.

```
In [9]: pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df2)
print(pca3.explained_variance_ratio_)
print(pca3.components_)
```

```
[0.86861085 0.09414617 0.03724298]
[[ 0.59031629  0.71908728  0.36666083]
 [ 0.32633701 -0.62809127  0.70640322]
 [ 0.73826204 -0.29734633 -0.6054373 ]]
```

The principal component vectors shown are orthogonal to each other (their dot product is basically zero). Here, we check that using the first two vectors.

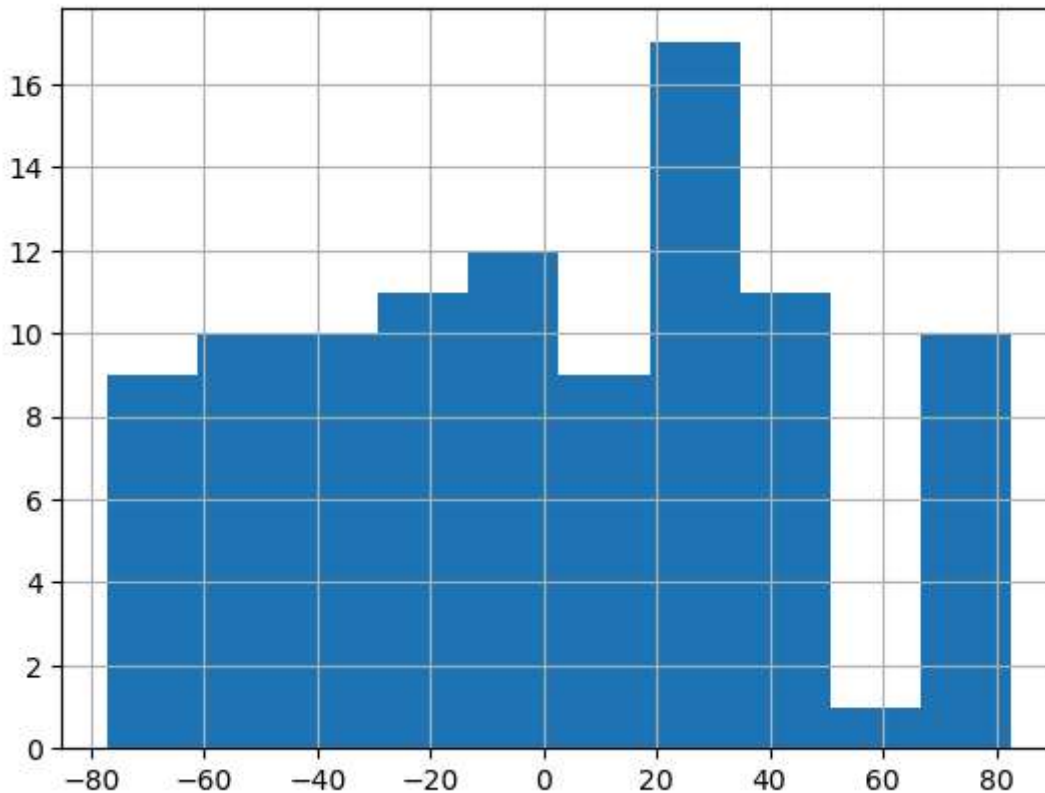
```
In [10]: print(np.array(pca3.components_[0]).dot(np.array(pca3.components_[1])))
```

```
1.1102230246251565e-16
```

We can use the projection onto the first principal component as if it were a new feature. For instance, we could plot a histogram of the data against the magnitude of the projection onto the first principal component. In fact, for some purposes, this magnitude (which we have called v) might make a better feature for data analysis than either height, width, or root. We must subtract off the mean before the dot product; the understanding is that this represents the component of the data relative to the mean.

```
In [11]: ((df2 - df2.mean()).dot(np.array(pca.components_[0]))).hist()
```

```
Out[11]: <Axes: >
```

```
In [12]: # Showing that this is the same as the value we've called "v"
(df_example["v"] - (df2 - df2.mean()).dot(np.array(pca.components_[0]))).max()
```

```
Out[12]: 0.0
```

Linear Regression

Now let's run a linear regression model to predict the root from the height and width, as well as predicting the third principal component from the first and second. Which is easier to predict?

Predicting the root from the height and width

```
In [13]: linreg = linear_model.LinearRegression()
linreg.fit(df2[['height', 'width']], df2['root'])
preds = linreg.predict(df2[['height', 'width']])
metrics.mean_squared_error(preds, df2['root']) # Mean squared error as a metric
```

```
Out[13]: 138.22487356329611
```

```
In [14]: np.corrcoef(preds, df2['root']) # Correlation coefficient as a metric
```

```
Out[14]: array([[1.          , 0.79468295],
                [0.79468295, 1.          ]])
```

Predicting the third principal component from the first and second

```
In [15]: first_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.components_[0]))
second_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.components_[1]))
third_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.components_[2]))
```

```
In [16]: linreg_first = linear_model.LinearRegression()
pc_X = np.array([first_principal_component_score, second_principal_component_score])
linreg_first.fit(pc_X, third_principal_component_score)
preds = linreg_first.predict(pc_X)
metrics.mean_squared_error(preds, third_principal_component_score) # The mean squared error
```

```
Out[16]: 78.74928905273175
```

```
In [17]: np.corrcoef(preds, third_principal_component_score) # More importantly, the correlation coefficient
```

```
Out[17]: array([[1.00000000e+00, 1.13873658e-15],
                [1.13873658e-15, 1.00000000e+00]])
```

Here, we see that the projections of the first two principal components have no ability to predict the projection of the third, when measured by correlation. This is a special property of principal components. It is intended to ensure that each component gives new information not present in the previous components.

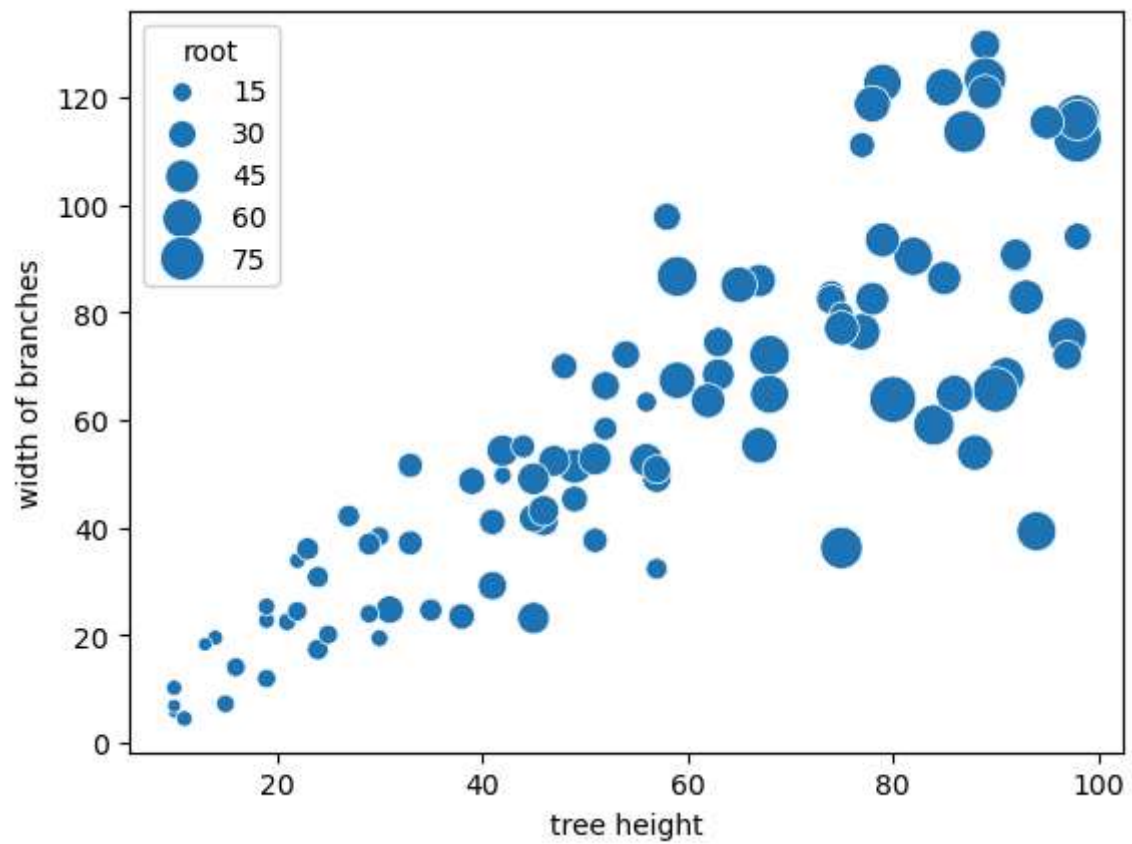
The mean_squared_error is misleading - it makes it look like the error is less for the second computation than the first; but the correlation coefficient is giving more useful information here. We cannot compare the mean squared error for two totally different computations, because mean squared error is highly dependent on the magnitudes of the values in question!

2. Weekly graph question

The tree data above can be graphed via either a pairplot or a bubble plot. What are the advantages and disadvantages of each?

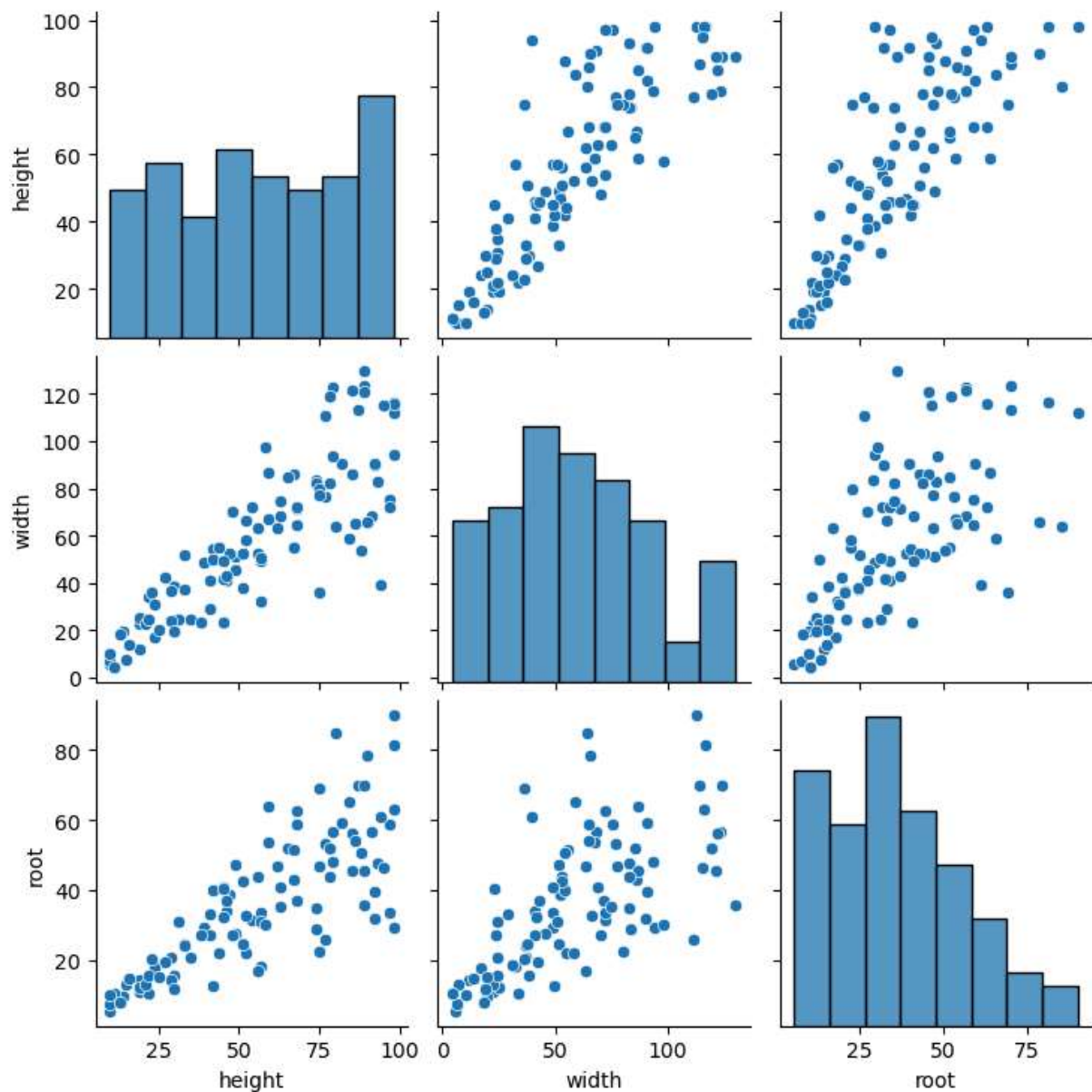
```
In [18]: ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "root", sizes = 1000)
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

```
Out[18]: Text(0, 0.5, 'width of branches')
```

```
In [19]: sns.pairplot(df2)
```

```
Out[19]: <seaborn.axisgrid.PairGrid at 0x12379419a50>
```



Plot Type	Pros	Cons
Pairplot	Shows relationships between different variables at once	Multiple information presented at once can be detrimental to a specific point trying to be made
Bubble plot	Highlights specific relationship between two variables	May lose context of underlying trends from other graphs

3. Homework - Grocery Store Correlation

In a particular state, the price of milk is highly correlated with the price of cheese. Neither is correlated with the price of bread.

```
In [20]: num_points = 1000
np.random.seed(0)
```

```

milk_series = np.maximum(np.random.normal(scale = 0.5, loc = 2.00, size = num_point
cheese_series = milk_series * np.maximum(np.random.normal(scale = 0.1, loc = 1.00,
bread_series = np.maximum(np.random.normal(scale = 1.0, loc = 3.00, size = num_poin
df3 = pd.DataFrame({"milk": milk_series, "cheese": cheese_series, "bread": bread_se

```

Heatmap

Draw a heatmap where the x-axis category is a discretized version of the milk price and the y-axis category is a discretized version of the cheese price. That is, you can choose "low", "medium", and "high" for the milk price, or, if you prefer, you can choose 1, 2, 3, 4, and 5 for the five quintiles. (The first quintile should include the bottom fifth of the data, the second quintile contains the next fifth, and so on).

Try Googling "how do I change numerical to categorical data" to get advice on this.

```

In [21]: milk_discretized = pd.qcut(milk_series, q = 5, labels = list(range(1,6))).astype(in
cheese_discretized = pd.qcut(cheese_series, q = 5, labels = list(range(1,6))).astyp
bread_discretized = pd.qcut(bread_series, q = 5, labels = list(range(1,6))).astype(

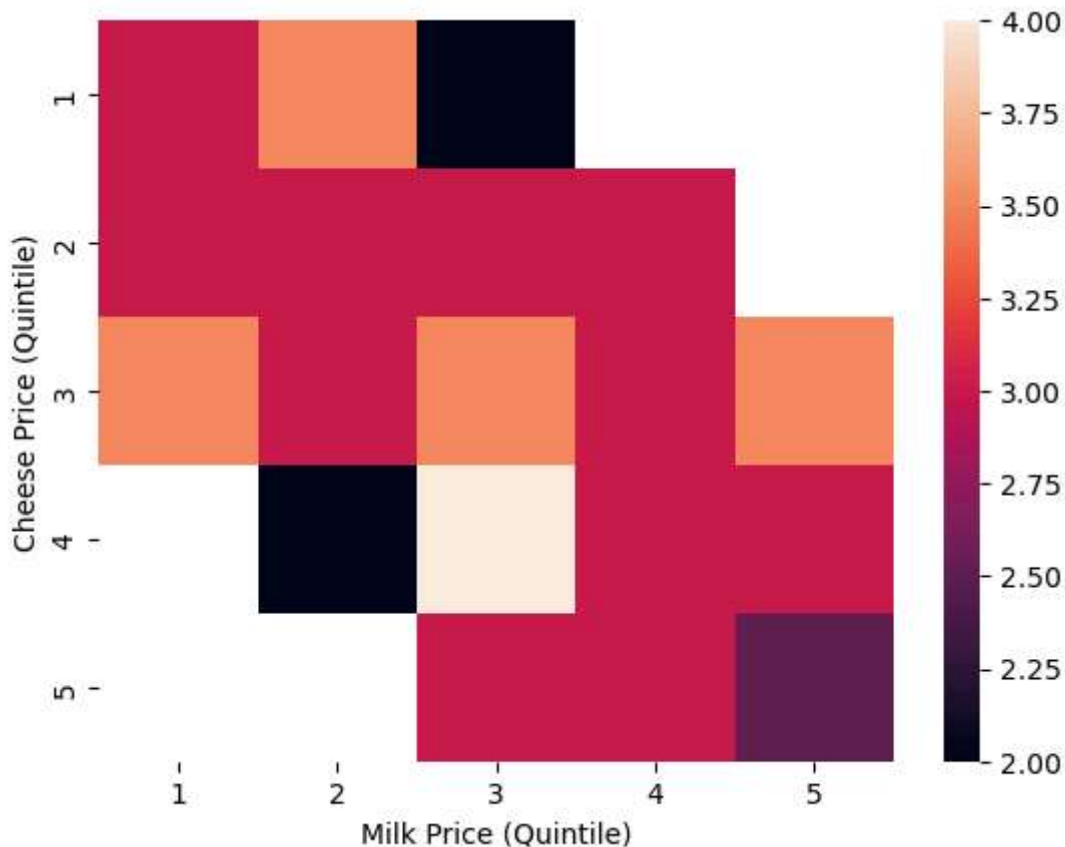
data = pd.DataFrame({'Milk Price (Quintile)': milk_discretized, 'Cheese Price (Quin
hdata = data.pivot_table(index = 'Cheese Price (Quintile)', columns = 'Milk Price (
sns.heatmap(hdata)

```

```

Out[21]: <Axes: xlabel='Milk Price (Quintile)', ylabel='Cheese Price (Quintile)'>

```

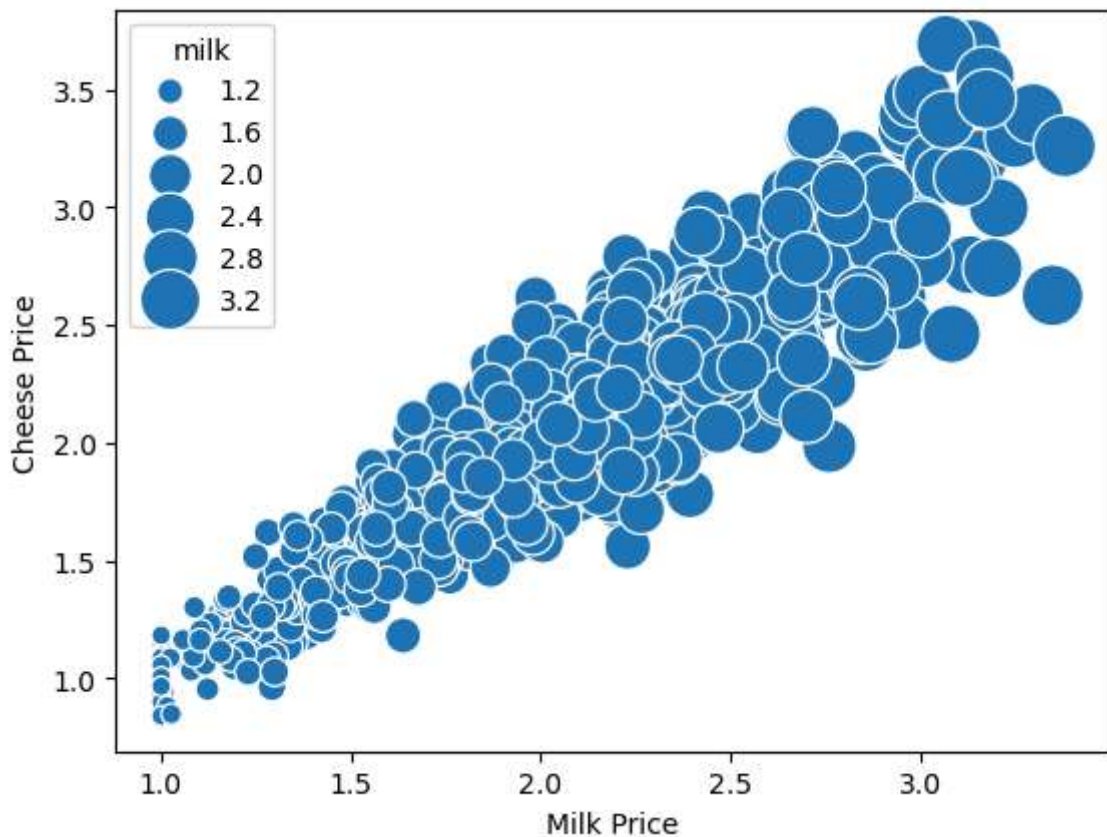


Bubble plot

Try all three possible bubble plots: one where the size of the dot is determined by the milk price, one where it's the cheese price, and one where it's the bread price. Which of these plots is / are the most informative and useful?

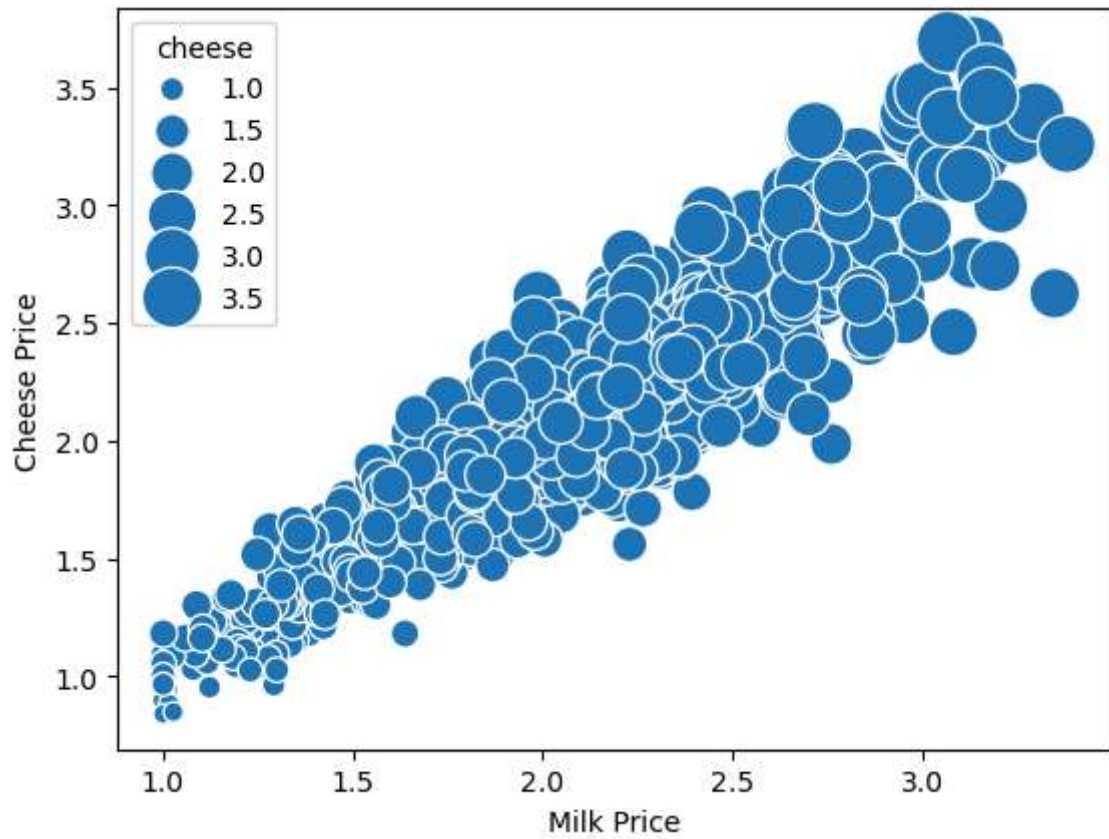
```
In [22]: ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size = "milk", sizes = (  
ax.set_xlabel("Milk Price")  
ax.set_ylabel("Cheese Price")
```

```
Out[22]: Text(0, 0.5, 'Cheese Price')
```



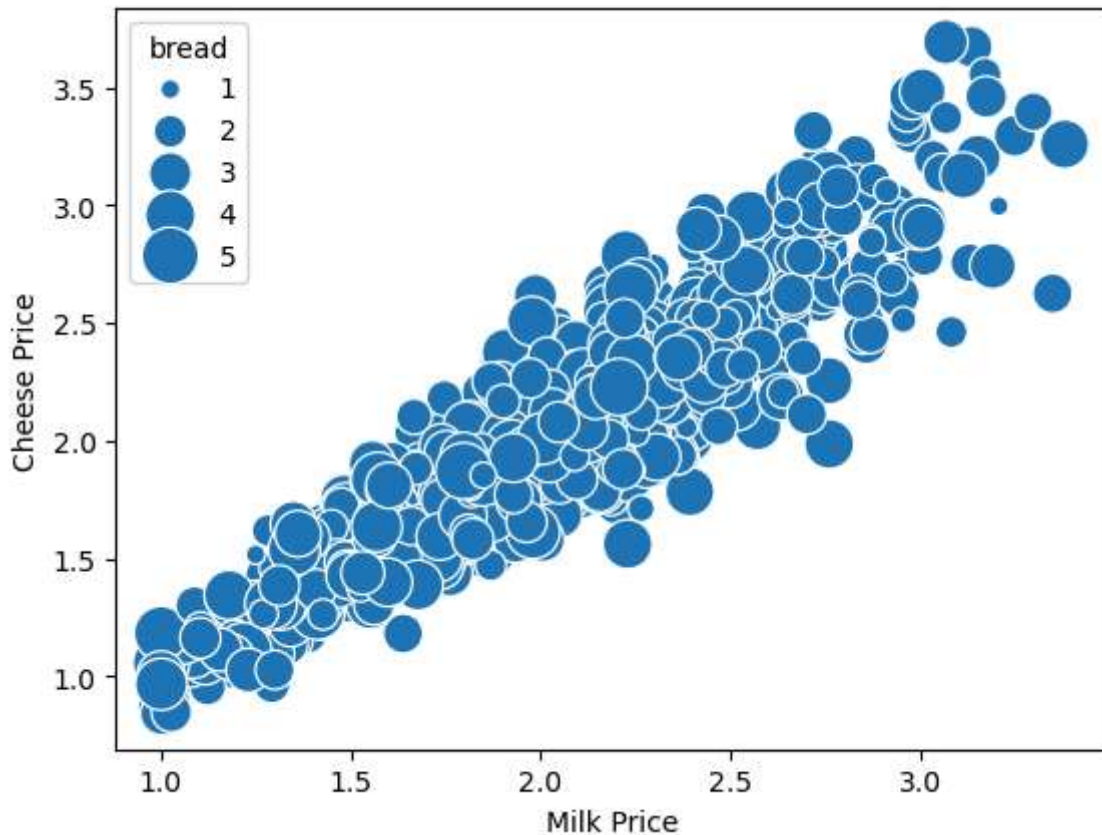
```
In [23]: ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size = "cheese", sizes = (  
ax.set_xlabel("Milk Price")  
ax.set_ylabel("Cheese Price")
```

```
Out[23]: Text(0, 0.5, 'Cheese Price')
```



```
In [24]: ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size = "bread", sizes =  
ax.set_xlabel("Milk Price")  
ax.set_ylabel("Cheese Price")
```

```
Out[24]: Text(0, 0.5, 'Cheese Price')
```

Principal component analysis

Perform a principal component analysis of the data. What are the first, second, and third component vectors?

Challenge question: explain why the three vectors point as they do. (Could someone have guessed the order of the three vectors, even before running the analysis?)

```
In [25]: model = decomposition.PCA(n_components = 3)
model.fit_transform(df3)
print(f"The three component vectors in dimensions (milk, cheese, bread price) are:
```

The three component vectors in dimensions (milk, cheese, bread price) are:

```
[[ -0.03825586 -0.03809452  0.99854158]
 [  0.67873842  0.73239615  0.05394468]
 [  0.73338301 -0.67981224  0.00216223]]
```

The three vectors point as they do because they represent the correlation between the three variables (bread price does not affect milk/cheese price, but milk and cheese influence each other). This could have been guessed before running the analysis.

Linear regression

Use linear regression to predict the milk price from the other two, the cheese price from the other two, and the bread price from the other two. Which prediction(s) work the best, and

why?

```
In [26]: from sklearn.model_selection import train_test_split
```

```
In [27]: model = linear_model.LinearRegression()
col = "milk"
X = df3.drop(col, axis = 1)
y = df3[col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Predicting {col} from the other two variables results in a MAE of {metrics.
```

Predicting milk from the other two variables results in a MAE of 0.15.

```
In [28]: model = linear_model.LinearRegression()
col = "cheese"
X = df3.drop(col, axis = 1)
y = df3[col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Predicting {col} from the other two variables results in a MAE of {metrics.
```

Predicting cheese from the other two variables results in a MAE of 0.16.

```
In [29]: model = linear_model.LinearRegression()
col = "bread"
X = df3.drop(col, axis = 1)
y = df3[col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Predicting {col} from the other two variables results in a MAE of {metrics.
```

Predicting bread from the other two variables results in a MAE of 0.85.

Predicting milk from the other two variables has the lowest MAE and thus performs the best due to least error.

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 120-135 of the Storytelling With Data book as best you can. ("Use color consistently" in chapter four up to "Create a clear visual hierarchy of information" in chapter five.) You do not have to get the exact data values right, just the overall look and feel.

```
In [30]: import matplotlib.pyplot as plt
```

```
In [31]: data = pd.DataFrame({
    'Categories': ['Category 1', 'Category 2', 'Category 3', 'Category 4', 'Categor
    'Values': [7, 5, 4, 4, 3]
```

```

}))

fig, ax = plt.subplots(figsize = (10,10))

bars = ax.barh(data['Categories'], data['Values'])

# Color bars and add text
for barnum, (bar, category, value) in enumerate(zip(bars, data['Categories'], data[
    x = bar.get_width()
    y = bar.get_y() + bar.get_height() / 2

    # Color bar
    color = 'gray' if barnum != 1 else 'green'
    bar.set_facecolor(color)

    # Add value text
    ax.text(x - 0.25, y, f"{value}", va = 'center', ha = 'left', color = 'white', f

# Flip order of bars
ax.invert_yaxis()

# Title
ax.text(0, 1.01, 'Leverage ', color = 'black', transform = ax.transAxes, fontsize =
ax.text(0.11, 1.01, 'brand color', color = 'green', transform = ax.transAxes, fonts

plt.show()

```

